EU AI Act Ontology — proof of concept

Author: Zhaoxing Li **Source of truth:** Regulation (EU) 2024/1689 (EUR-Lex, CELEX 32024R1689) **Version:** 0.1.0

1. Scope

I built a proof-of-concept ontology of the core of the EU AI Act — not a complete or publication-quality model. It covers the elements the task asks for: AI system categories, prohibited practices, high-risk systems, the core actors (providers, deployers, importers, distributors, authorised representatives, and a market surveillance authority), the Annex III high-risk domains, and the major obligations. Every modelled element carries a label, a short definition, an article or annex reference, and a verbatim snippet of the source provision. I also keep the risk-tier vocabulary used to organise the Act — `UnacceptableRisk` and `HighRisk` are article-grounded, while `LimitedRisk` and `MinimalRisk` are common usage (recital-level) and are tagged as such.

Deliberately out of scope: general-purpose AI model obligations beyond a stub class, conformity-assessment and notified-body procedures, penalties, the full exception structure of conditional provisions (Article 5(1)(h), Article 50(4)), and recital detail. Both high-risk routes are represented — the Annex III route (Article 6(2)) as a defined class and the product-safety route (Article 6(1)) as a stub, since the latter depends on the Annex I product legislation I do not model. My guiding rule was a small, axiomatically rich core over a large, thin taxonomy.

2. AI-assisted pipeline

I used a large language model (Claude, in an agentic coding environment) at three points, each fenced by an accuracy control I built around it:

1. **Fetch + segment** (`fetch_sources.py`): download the official EUR-Lex HTML and split it into 126 article/annex segments, each tagged with its identifier.
2. **AI extraction** (`extract_fragments.py`): the model proposed candidate classes, actors, obligations, practices and domains. Each candidate must cite a **verbatim anchor phrase**; the script re-locates it in the source and copies the surrounding span. Anything not locatable verbatim is auto-flagged.
3. **Illustrative conformal-style review gate** (`score_extractions.py`): each candidate gets an interpretable confidence score; a split-conformal threshold decides which are auto-accepted and which go to a human. Under the stated (small, partly synthetic) calibration setup this gives a distribution-free *coverage* statement, tunable by α — a proof of concept, not a guarantee of legal correctness. The gate recovers the hand-set review flags as α is tuned (see `docs/confidence-gating.md`). I then decided every routed candidate (`verification-table.md`) before authoring.
4. **AI authoring** (`build_ontology.py`): I assembled the verified candidates into Turtle with full provenance and the hand-designed axiomatic core.
5. **Reason + test + back-check** (`build_checks.py`, `verify_provenance.py`).

How I kept it accurate: source-grounded extraction (no concept without a span), strict flagging of paraphrase/conditional/recital items, a human gate, and a final hallucination back-check that re-matches every snippet in the finished ontology against the source text (**47/47 matched, 0 unmatched**). The model proposes; I decide what the law means. As a creative extension I added an adversarial **Extractor/Modeller/Critic** variant (`docs/multi-agent.md`) where the Critic attacks each axiom with the source text; its flags converge with the heuristic and conformal gate, and it surfaced one issue the others missed (the Article 6(1) high-risk route) — which I then acted on by modelling both routes.

3. Key modelling decisions

- **Reuse (AIRO).** I align classes to the AI Risk Ontology (`https://w3id.org/airo#`): `AISystem ≡ airo:AISystem`, `Actor ≡ airo:AIOperator`, `Provider ⊑ airo:AIProvider`, `Deployer ⊑ airo:AIDeployer`, `AnnexIIIDomain ⊑ airo:Domain`, `operatesInDomain ⊑ airo:isAppliedWithinDomain`. I inspected the AIRO class set before aligning rather than guessing. **Deliberate non-reuse:** the Act's risk *tiers* are local `RiskLevel` individuals and are *not* aligned to `airo:Risk`, which is a quantitative likelihood×severity notion, not a regulatory category.
- **Risk is reasoned; high-risk follows Article 6's two routes.** Rather than a parallel risk-category subclass tree, `AnnexIIIHighRiskAISystem` is a defined class — `AISystem ⊓ (operatesInDomain some AnnexIIIDomain)` (Art 6(2)); `ProductSafetyHighRiskAISystem` is the Art 6(1) stub; and `HighRiskAISystem` is their union. A reasoner *infers* membership. Two demo systems assert only their Annex III domain and are classified high-risk; a chatbot with no Annex III domain is correctly left limited-risk (no over-classification).
- **Obligations typed by kind; bearer by property.** Obligation subclasses are by kind (transparency, risk management, data governance, technical documentation, logging, human oversight, post-market monitoring, accuracy/robustness). The bearer is attached with `imposedOn` and the target system kind with `appliesToSystem` (OWL 2 punning), avoiding a one-dimensional tree that mixes bearer and type.
- **Provenance.** PROV-O on the ontology header records the pipeline; every element carries article/annex references and a `legalTextSnippet`.

4. Competency questions and example SPARQL

CQ1 prohibited practices + article (Art 5) · CQ2 provider obligations for high-risk systems (Arts 9–15, 72) · CQ3 actors bearing high-risk obligations — provider, deployer, importer, distributor, authorised representative (Arts 16, 22, 23, 24, 26) · CQ4 systems *inferred* high-risk and their Annex III domain (Art 6(2)) · CQ5 transparency obligations for limited-risk systems (Art 50). All five return non-empty, correct results on the reasoned graph (`queries/results/`).

```
# CQ4 — relies on inference; no system is asserted as high-risk
PREFIX euai: <https://zhaoxing-li.github.io/eu-ai-act-ontology#>
SELECT ?system ?domain WHERE {
  ?system a euai:HighRiskAISystem ; euai:operatesInDomain ?domain . }
```

```
# CQ1 — provenance is queryable
PREFIX euai: <https://zhaoxing-li.github.io/eu-ai-act-ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?practice ?label ?article WHERE {
  ?practice a euai:ProhibitedPractice ; rdfs:label ?label ;
            euai:hasArticleReference ?article . }
```

5. Validation, metrics and further evaluation

Validation. The ontology parses with `rdflib`, is **consistent under HermiT** with no unsatisfiable classes, and loads and classifies in Protégé (`docs/protege-validation.md`). The headline inference (two systems → high-risk; chatbot → not) holds.

Preliminary metrics. 30 classes · 6 object + 3 annotation properties · 38 named individuals · 4 logically-defined classes (incl. the Annex III restriction and the high-risk union) · 11 AIRO alignment axioms · 51 article + 9 annex references · 457 asserted → 493 reasoned triples · CQ coverage 5/5 · snippet match 47/47.

Proposed further evaluation. (1) **CQ coverage** expanded to a larger, expert-authored question set; (2) **OOPS!** pitfall scan for common modelling errors; (3) **expert spot-check** of a random sample of element-to-article mappings by a legal/SW expert, reporting precision; (4) **competency-question recall** against an independent reading of the Act; (5) **reuse ratio** and FAIR/FOOPS! assessment of metadata completeness.

6. Why I built it this way

My background is in LLM-based multi-agent systems and trustworthy, human-centric AI; formal RDF/OWL is the area I have been deliberately growing into, and this task was a chance to take it end to end. That shaped the choices here: I leaned on my strength with generative and agentic AI to move fast (an LLM does the extraction, an adversarial Critic agent checks it), but I kept the reasoning in the symbolic layer — a defined class the reasoner classifies, principled AIRO reuse, and provenance on every element — so the result is verifiable rather than just fluent. That neuro-symbolic split, and the habit of pairing generation with verification, is how I would approach building trustworthy information resources in the group.